

Gemini 3.8 Flash is now available. [Try it out](#)

(https://aistudio.google.com/prompts/new_chat?model=gemini-3.8-flash).

Generate videos with Veo 3.1 in Gemini API



To learn about video understanding, see the [Video understanding](#) (/gemini-api/docs/video-understanding) guide.

[Veo 3.1](#) (<https://deepmind.google/models/veo/>) is a model for generating 8-second videos (720p, 1080p, or 4k) with natively generated audio. You can access this model programmatically using the Gemini API. To learn more about the available Veo model variants, see the [Model Versions](#) (#model-versions) section.

Veo 3.1 excels at a wide range of visual and cinematic styles and introduces several new capabilities:

- **Portrait videos:** Choose between landscape (16:9) and portrait (9:16) videos.
- **Video extension:** Extend videos that were previously generated using Veo.
- **Frame-specific generation:** Generate a video by specifying the first and last frames.
- **Image-based direction:** Use up to three reference images to guide the content of your generated video.

For more information about writing effective text prompts for video generation, see the [Veo prompt guide](#) (#prompt-guide)

Text to video generation

The following examples show how you can generate a video with [dialogue](#) (#dialogue), [cinematic realism](#) (#realism), or [creative animation](#) (#style):

Dialogue & sound effects

```

import time
from google import genai
from google.genai import types

client = genai.Client()

prompt = """A close up of two people staring at a cryptic drawing on a
wall, torchlight flickering.
A man murmurs, 'This must be it. That's the secret code.' The woman
looks at him and whispering excitedly, 'What did you find?'''

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the generated video.
generated_video = operation.response.generated_videos[0]
client.files.download(file=generated_video.video,
destination="dialogue_example.mp4")
print("Generated video saved to dialogue_example.mp4")

```



Veo 3: Uncovering the Secret Code in a Cryptic Drawing

Google for Developers



Watch on

Cinematic realism

[Python](#) [JavaScript](#) (#javascript) [Go](#) (#go) [Java](#) (#java) [REST](#) (#rest)
(#python)

```
import time
from google import genai
from google.genai import types

client = genai.Client()

prompt = """Drone shot following a classic red convertible driven by a
man along a winding coastal road at sunset, waves crashing against the
rocks below.
The convertible accelerates fast and the engine roars loudly."""

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the generated video.
generated_video = operation.response.generated_videos[0]
client.files.download(file=generated_video.video,
destination="realism_example.mp4")
print("Generated video saved to realism_example.mp4")
```



Veo 3: Epic Drone Shot Following a Roaring Convertible at Sur

Google for Developers





Watch on

Creative animation

[Python](#) [JavaScript](#) (#javascript) [Go](#) (#go) [Java](#) (#java) [REST](#) (#rest) [\(#python\)](#)

```
import time
from google import genai

client = genai.Client()
prompt = "A whimsical stop-motion animation of a tiny robot tending to a garden of glowing mushrooms on a miniature planet."

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the generated video.
generated_video = operation.response.generated_videos[0]
client.files.download(file=generated_video.video,
    destination="style_example.mp4")
print("Generated video saved to style_example.mp4")
```



Veo 3: The Tiny Robot and His Glowing Mushroom Garden

Google for Developers





Watch on

Control the aspect ratio

Veo 3.1 lets you create landscape (16:9, the default setting) or portrait (9:16) videos. You can tell the model which one you want using the `aspect_ratio` parameter:

[Python](#)[JavaScript](#) (#javascript)[Go](#) (#go)[REST](#) (#rest)
(#python)

```
import time
from google import genai
from google.genai import types

client = genai.Client()

prompt = """A montage of pizza making: a chef tossing and flattening the
floury dough, ladling rich red tomato sauce in a spiral, sprinkling
mozzarella cheese and pepperoni, and a final shot of the bubbling
golden-brown pizza, upbeat electronic music with a rhythmical beat is
playing, high energy professional video."""

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
    config=types.GenerateVideosConfig(
        aspect_ratio="9:16",
    ),
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the generated video.
generated_video = operation.response.generated_videos[0]
client.files.download(file=generated_video.video,
```

```
destination="pizza_making.mp4")
print("Generated video saved to pizza_making.mp4")
```



Veo 3.1 Demo videos

Google for Developers



Watch on

Control the resolution

Veo 3.1 can also directly generate 720p, 1080p or 4k videos (4k not available for Veo 3.1 Lite).

Note that the higher the resolution, the higher the latency will be. 4k videos are also more expensive (see [pricing](#) (/gemini-api/docs/pricing#veo-3.1)).

[Video extension](#) (#extending_veo_videos) is also limited to 720p videos.

[Python](#)[JavaScript](#) (#javascript)[Go](#) (#go)[REST](#) (#rest)
(#python)

```
import time
from google import genai
from google.genai import types

client = genai.Client()

prompt = """A stunning drone view of the Grand Canyon during a
flamboyant sunset that highlights the canyon's colors. The drone slowly
flies towards the sun then accelerates, dives and flies inside the
```

```

canyon. """

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
    config=types.GenerateVideosConfig(
        resolution="4k",
    ),
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the generated video.
generated_video = operation.response.generated_videos[0]
client.files.download(file=generated_video.video,
    destination="4k_grand_canyon.mp4")
print("Generated video saved to 4k_grand_canyon.mp4")

```



Veo 3.1 Demo videos
Google for Developers



Watch on

Image to video generation

The following code demonstrates generating an image using [Gemini 3.1 Flash Image aka Nano Banana 2](#) (/gemini-api/docs/image-generation), then using that image as the starting

frame for generating a video with Veo 3.1.

PythonJavaScript (#javascript)Go (#go)Java (#java)
(#python)

```
import time
from google import genai

client = genai.Client()

prompt = "Panning wide shot of a calico kitten sleeping in the sunshine"

# Step 1: Generate an image with Nano Banana 2.
image = client.models.generate_content(
    model="gemini-3.1-flash-image-preview",
    contents=prompt,
    config={"response_modalities": ['IMAGE']}
)

# Step 2: Generate video with Veo 3.1 using the image.
operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
    image=image.parts[0].as_image(),
)

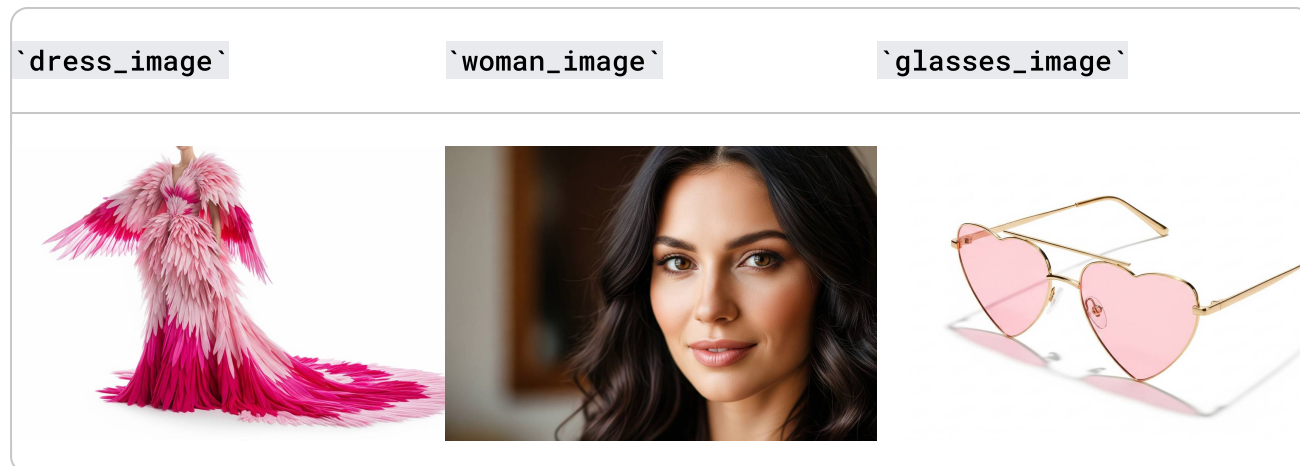
# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the video.
video = operation.response.generated_videos[0]
client.files.download(file=video.video,
    destination="veo3_with_image_input.mp4")
print("Generated video saved to veo3_with_image_input.mp4")
```

Using reference images

Veo 3.1 now accepts up to 3 reference images to guide your generated video's content. Provide images of a person, character, or product to preserve the subject's appearance in the output video.

For example, using these three images generated with [Nano Banana](#) (/gemini-api/docs/image-generation) as references with a [well-written prompt](#) (#use-reference-images) creates the following video:



[Python](#)[JavaScript](#) (#javascript)[Go](#) (#go)[REST](#) (#rest)
(#python)

```
import time
from google import genai
```

```
client = genai.Client()
```

```
prompt = "The video opens with a medium, eye-level shot of a beautiful woman with dark hair and warm brown eyes. She wears a magnificent, high-fashion flamingo dress with layers of pink and fuchsia feathers, complemented by whimsical pink, heart-shaped sunglasses. She walks with serene confidence through the crystal-clear, shallow turquoise water of a sun-drenched lagoon. The camera slowly pulls back to a medium-wide shot, revealing the breathtaking scene as the dress's long train glides and floats gracefully on the water's surface behind her. The cinematic, dreamlike atmosphere is enhanced by the vibrant colors of the dress against the serene, minimalist landscape, capturing a moment of pure elegance and high-fashion fantasy."
```

```
dress_reference = types.VideoGenerationReferenceImage(
    image=dress_image, # Generated separately with Nano Banana
    reference_type="asset"
)
```

```
sunglasses_reference = types.VideoGenerationReferenceImage(
    image=glasses_image, # Generated separately with Nano Banana
    reference_type="asset"
)
```

```
woman_reference = types.VideoGenerationReferenceImage(
```

```

    image=woman_image, # Generated separately with Nano Banana
    reference_type="asset"
)

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
    config=types.GenerateVideosConfig(
        reference_images=[dress_reference, glasses_reference,
woman_reference],
    ),
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the video.
video = operation.response.generated_videos[0]
client.files.download(file=video.video,
destination="veo3.1_with_reference_images.mp4")
print("Generated video saved to veo3.1_with_reference_images.mp4")

```



Flamingo fashion cat-walk in a sun-drenched lagoon

Google for Developers



Watch on

Using first and last frames

Veo 3.1 lets you create videos using interpolation, or specifying the first and last frames of the video. For information about writing effective text prompts for video generation, see the [Veo prompt guide](#) (#use-reference-images).

PythonJavaScript (#javascript)**Go** (#go)**REST** (#rest)
(#python)

```
import time
from google import genai

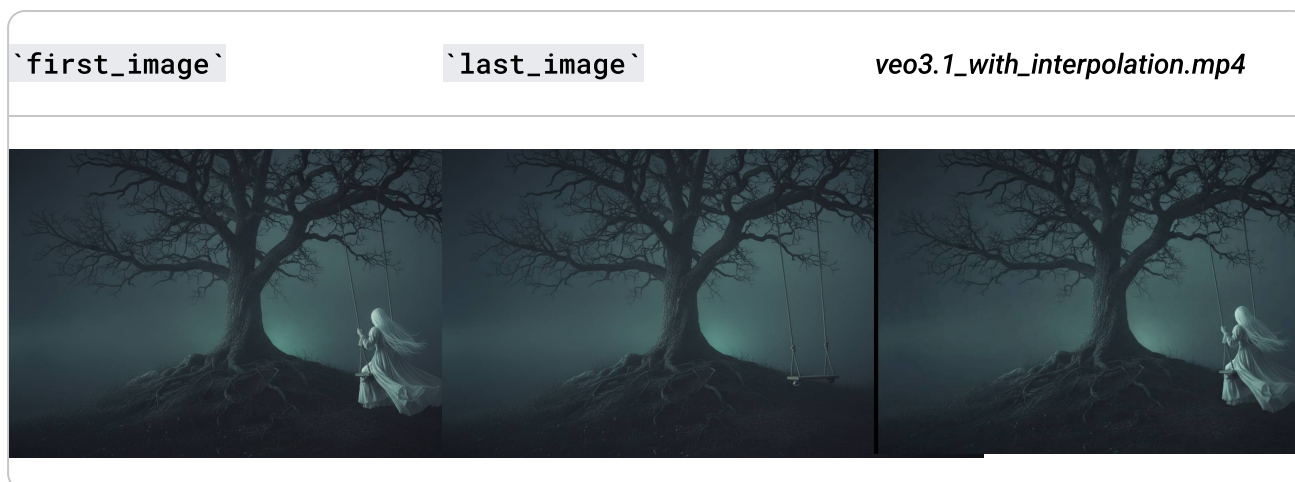
client = genai.Client()

prompt = "A cinematic, haunting video. A ghostly woman with long white hair and a flowing dress swings gently on a rope swing beneath a massive, gnarled tree in a foggy, moonlit clearing. The fog thickens and swirls around her, and she slowly fades away, vanishing completely. The empty swing is left swaying rhythmically on its own in the eerie silence."

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt=prompt,
    image=first_image, # The starting frame is passed as a primary input
    config=types.GenerateVideosConfig(
        last_frame=last_image # The ending frame is passed as a generation
                             # constraint in the config
    ),
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the video.
video = operation.response.generated_videos[0]
client.files.download(file=video.video,
    destination="veo3.1_with_interpolation.mp4")
print("Generated video saved to veo3.1_with_interpolation.mp4")
```



Extending Veo videos

Use Veo 3.1 to extend videos that you previously generated with Veo by 7 seconds and up to 20 times.

Input video limitations:

- Veo-generated videos only up to 141 seconds long.
- Gemini API only supports video extensions for Veo-generated videos.
- The video should come from a previous generation, like `operation.response.generated_videos[0].video`
- Videos are stored for 2 days, but if a video is referenced for extension, its 2-day storage timer resets. You can only extend videos that were generated or referenced in the last two days.
- Input videos are expected to have a certain length, aspect ratio, and dimensions:
 - Aspect ratio: 9:16 or 16:9
 - Resolution: 720p
 - Video length: 141 seconds or less

The output of the extension is a single video combining the user input video and the generated extended video for up to 148 seconds of video.

This example takes the a Veo-generated video, shown here with its original prompt, and extends it using the `video` parameter and a new prompt:

Prompt

Output: butterfly_video

An origami butterfly flaps its wings and flies out of the french doors into the garden.



PythonJavaScript (#javascript)Go (#go)REST (#rest)
(#python)

```
import time
from google import genai

client = genai.Client()

prompt = "Track the butterfly into the garden as it lands on an orange
origami flower. A fluffy white puppy runs up and gently pats the
flower."

operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    video=operation.response.generated_videos[0].video, # This must be a
video from a previous generation
    prompt=prompt,
    config=types.GenerateVideosConfig(
        number_of_videos=1,
        resolution="720p"
    ),
)

# Poll the operation status until the video is ready.
while not operation.done:
    print("Waiting for video generation to complete...")
    time.sleep(10)
    operation = client.operations.get(operation)

# Download the video.
video = operation.response.generated_videos[0]
client.files.download(file=video.video,
destination="veo3.1_extension.mp4")
print("Generated video saved to veo3.1_extension.mp4")
```



A beautiful day for origami and puppies!

Google for Developers



Watch on

For information about writing effective text prompts for video generation, see the [Veo prompt guide](#) (#extend-prompt).

Handling asynchronous operations

Video generation is a computationally intensive task. When you send a request to the API, it starts a long-running job and immediately returns an `operation` object. You must then poll until the video is ready, which is indicated by the `done` status being true.

The core of this process is a polling loop, which periodically checks the job's status.

[Python](#) [JavaScript](#) (#javascript) [Go](#) (#go) [Java](#) (#java) [REST](#) (#rest)
(#python)

```
import time
from google import genai
from google.genai import types

client = genai.Client()

# After starting the job, you get an operation object.
operation = client.models.generate_videos(
    model="veo-3.1-generate-preview",
    prompt="A cinematic shot of a majestic lion in the savannah.",
)
```

```
# Alternatively, you can use operation.name to get the operation.
operation = types.GenerateVideosOperation(name=operation.name)

# This loop checks the job status every 10 seconds.
while not operation.done:
    time.sleep(10)
    # Refresh the operation object to get the latest status.
    operation = client.operations.get(operation)

# Once done, the result is in operation.response.
# ... process and download your video ...
```

Veo API parameters and specifications

These are the parameters you can set in your API request to control the video generation process.

Parameter	Veo 3.1 & Veo 3.1 Fast	Veo 3.1 Lite	Veo 3 & Veo 3 Fast
<i>Instances</i>			
prompt: The text description for the video. Supports audio cues.	string	string	string
image: An initial image to animate.	Image object	Image object	Image object
lastFrame: The final image for an interpolation video to transition. Must be used in combination with the image parameter.	Image object	Image object	Image object
referenceImages: Up to three images to be used as style and content references.	VideoGeneration ReferenceImage object	n/a object	n/a
video: Video to be used for video	Video object from a previous generation	n/a	n/a

Parameter	Veo 3.1 & Veo 3.1 Fast	Veo 3.1 Lite	Veo 3 & Veo 3 Fast
extension.			
<i>Parameters</i>			
aspectRatio: The video's aspect ratio.	"16:9" (default), "9:16"	"16:9" (default), "9:16"	"16:9" (default), "9:16"
durationSeconds: Length of the generated video.	"4", "6", "8". <i>Must be "8" when using extension, reference images or with 1080p and 4k resolutions</i>	"4", "6", "8". <i>Must be "8" when using reference images or with 1080p</i>	"4", "6", "8". <i>Must be "8" when using extension, reference images or with 1080p and 4k resolutions</i>
personGeneration: Controls the generation of people. (See Limitations (#limitations) for region restrictions)	Text-to-video & Extension: "allow_all" only Image-to-video, Interpolation, & Reference images: "allow_adult" only	Text-to-video: "allow_all" only Image-to-video, Interpolation, & Reference images: "allow_adult" only	Text-to-video: "allow_all" only Image-to-video: "allow_adult" only
resolution: The video's resolution.	"720p" (default), "1080p" (only supports 8s duration), "4k" (only supports 8s duration) "720p" only for extension	"720p" (default), "1080p" (only supports 8s duration)	"720p" (default), "1080p" (only supports 8s duration), "4k" (only supports 8s duration) "720p" only for extension

Note that the `seed` parameter is also available for Veo 3 models. It doesn't guarantee determinism, but slightly improves it.

Model features

Feature	Veo 3.1 & Veo 3.1 Fast	Veo 3.1 Lite	Veo 3 & Veo 3 Fast
Audio: Natively generates audio with video.	✓ Always on	✓ Always on	✓ Always on
Input modalities: The type of input used for generation.	Text-to-Video, Image-to-Video, Video-to-Video		
Resolution: The output resolution of the video.	720p, 1080p (8s length only), 4k (8s length only) <i>720p only when using video extension.</i>	720p, 1080p (8s length only)	720p & 1080p (16:9 only)
Frame rate: The output frame rate of the video.	24fps	24fps	24fps
Video duration: Length of the generated video.	8 seconds, 6 seconds, 4 seconds <i>8 seconds only if 1080p or 4k or using reference images</i>	8 seconds, 6 seconds, 4 seconds <i>8 seconds only if 1080p or using reference images</i>	8 seconds
Videos per request: Number of videos generated per request.	1	1	1
Status: Model availability	<u>Preview</u> (/gemini-api/docs/models#preview)	<u>Preview</u> (/gemini-api/docs/models#preview)	<u>Stable</u> (/gemini-api/docs/models#stable)

Limitations

- **Multi-video prompting:** Referencing or reasoning across multiple videos is not currently supported. Attempting multi-video prompting may result in degraded model performance or unexpected outputs.
- **Language support:** English (EN) is fully supported, but other languages have not been evaluated, so they may work but results can vary.
- **Request latency:** Min: 11 seconds; Max: 6 minutes (during peak hours).
- **Regional limitations:** In EU, UK, CH, MENA locations, `allow_adult` is the only allowed value for `personGeneration`.
- **Video retention:** Generated videos are stored on the server for 2 days, after which they are removed. To save a local copy, you must download your video within 2 days of generation. Extended videos are treated as newly generated videos.
- **Watermarking:** Videos created by Veo are watermarked using SynthID (<https://deepmind.google/technologies/synthid/>), our tool for watermarking and identifying AI-generated content. Videos can be verified using the SynthID (<https://deepmind.google/science/synthid/>) verification platform.
- **Safety:** Generated videos are passed through safety filters and memorization checking processes that help mitigate privacy, copyright and bias risks.
- **Audio error:** Veo 3.1 will sometimes block a video from generating because of safety filters or other processing issues with the audio. You will not be charged if your video is blocked from generating.

Veo prompt guide

This section contains examples of videos you can create using Veo, and shows you how to modify prompts to produce distinct results.

Safety filters

Veo applies safety filters across Gemini to help ensure that generated videos and uploaded photos don't contain offensive content. Prompts that violate our [terms and guidelines](#) ([/gemini-api/docs/usage-policies#abuse-monitoring](#)) are blocked.

Prompt writing basics

Good prompts are descriptive and clear. To get the most out of Veo, start with identifying your core idea, refine your idea by adding keywords and modifiers, and incorporate video-specific terminology into your prompts.

The following elements should be included in your prompt:

- **Subject:** The object, person, animal, or scenery that you want in your video, such as *cityscape, nature, vehicles, or puppies*.
- **Action:** What the subject is doing (for example, *walking, running, or turning their head*).
- **Style:** Specify creative direction using specific film style keywords, such as *sci-fi, horror film, film noir*, or animated styles like *cartoon*.
- **Camera positioning and motion:** [Optional] Control the camera's location and movement using terms like *aerial view, eye-level, top-down shot, dolly shot, or worms eye*.
- **Composition:** [Optional] How the shot is framed, such as *wide shot, close-up, single-shot or two-shot*.
- **Focus and lens effects:** [Optional] Use terms like *shallow focus, deep focus, soft focus, macro lens*, and *wide-angle lens* to achieve specific visual effects.
- **Ambiance:** [Optional] How the color and light contribute to the scene, such as *blue tones, night, or warm tones*.

More tips for writing prompts

- **Use descriptive language:** Use adjectives and adverbs to paint a clear picture for Veo.
- **Enhance the facial details:** Specify facial details as a focus of the photo like using the word *portrait* in the prompt.

For more comprehensive prompting strategies, visit [Introduction to prompt design](https://gemini-api/docs/prompting-intro) (/gemini-api/docs/prompting-intro).

Prompting for audio

You can provide Veo with cues for sound effects, ambient noise, and dialogue. The model captures the nuance of these cues to generate a synchronized soundtrack.

- **Dialogue:** Use quotes for specific speech. (Example: "This must be the key," he murmured.)

- **Sound Effects (SFX):** Explicitly describe sounds. (Example: tires screeching loudly, engine roaring.)
- **Ambient Noise:** Describe the environment's soundscape. (Example: A faint, eerie hum resonates in the background.)

These videos demonstrate prompting Veo 3's audio generation with increasing levels of detail.

Prompt	Generated output
More detail (Dialogue and ambience) A wide shot of a misty Pacific Northwest forest. Two exhausted hikers, a man and a woman, push through ferns when the man stops abruptly, staring at a tree. Close-up: Fresh, deep claw marks are gouged into the tree's bark. Man: (Hand on his hunting knife) "That's no ordinary bear." Woman: (Voice tight with fear, scanning the woods) "Then what is it?" A rough bark, snapping twigs, footsteps on the damp earth. A lone bird chirps.	
Less detail (Dialogue) Paper Cut-Out Animation. New Librarian: "Where do you keep the forbidden books?" Old Curator: "We don't. They keep us."	

Try out these prompts yourself to hear the audio! [Try Veo \(https://deepmind.google/models/veo/\)](https://deepmind.google/models/veo/)

Prompting with reference images

You can use one or more images as inputs to guide your generated videos, using Veo's [image-to-video \(/gemini-api/docs/veo#generate-from-images\)](https://deepmind.google/models/veo/) capabilities. Veo uses the input image as the initial frame. Select an image closest to what you envision as the first scene of your video to animate everyday objects, bring drawings and paintings to life, and add movement and sound to nature scenes.

Prompt

Generated output

Input image (Generated by Nano Banana)

A hyperrealistic macro photo of tiny, miniature surfers riding ocean waves inside a rustic stone bathroom sink. A vintage brass faucet is running, creating the perpetual surf. Surreal, whimsical, bright natural lighting.



Output Video (Generated by Veo 3.1)

A surreal, cinematic macro video. Tiny surfers ride perpetual, rolling waves inside a stone bathroom sink. A running vintage brass faucet generates the endless surf. The camera slowly pans across the whimsical, sunlit scene as the miniature figures expertly carve the turquoise water.



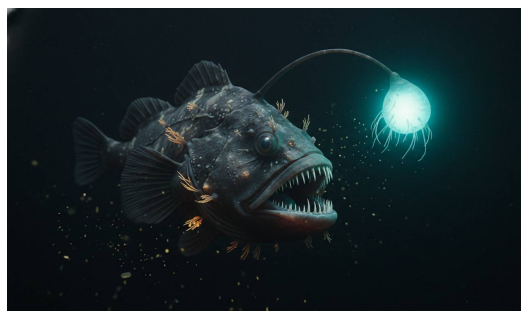
Veo 3.1 lets you [reference images](/gemini-api/docs/veo#reference-images) or ingredients to direct your generated video's content. Provide up to three asset images of a single person, character, or product. Veo preserves the subject's appearance in the output video.

Prompt

Generated output

Reference image (Generated by Nano Banana)

A deep sea angler fish lurks in the deep dark water, teeth bared and bait glowing.



Reference image (Generated by Nano Banana)

A pink child's princess costume complete with a wand and tiara, on a plain product background.



Prompt	Generated output
Output Video (Generated by Veo 3.1) Create a silly cartoon version of the fish wearing the costume, swimming and waving the wand around.	

Using Veo 3.1, you can also generate videos by specifying the first and last frames (/gemini-api/docs/veo#using-first-and-last-video-frames) of the video.

Prompt	Generated output
First image (Generated by Nano Banana) A high quality photorealistic front image of a ginger cat driving a red convertible racing car on the French riviera coast.	
Last image (Generated by Nano Banana) Show what happens when the car takes off from a cliff.	
Output Video (Generated by Veo 3.1) Optional	

This feature gives you precise control over your shot's composition by letting you define the starting and ending frame. Upload an image or use a frame from a previous video generation to make sure your scene begins and concludes exactly as you envision it.

Prompting for extension

To [extend](/gemini-api/docs/veo#extending-veo-videos) (/gemini-api/docs/veo#extending-veo-videos) your Veo-generated video with Veo 3.1 (not available for Veo 3.1 Lite), use the video as an input along with an optional text prompt. Extend finalizes the final second or 24 frames of your video and continues the action.

Note that voice is not able to be effectively extended if it's not present in the last 1 second of video.

Prompt	Generated output
Input video (Generated by Veo 3.1) The paraglider takes off from the top of the mountain and starts gliding down the mountains overlooking the flower covered valleys below.	
Output Video (Generated by Veo 3.1) Extend this video with the paraglider slowly descending.	

Example prompts and output

This section presents several prompts, highlighting how descriptive details can elevate the outcome of each video.

Icicles

This video demonstrates how you can use the elements of [prompt writing basics](#) (#basics) in your prompt.

Prompt

Generated output

Close up shot (composition) of melting icicles (subject) on a frozen rock wall (context) with cool blue tones (ambiance), zoomed in (camera motion) maintaining close-up detail of water drips (action).



Man on the phone

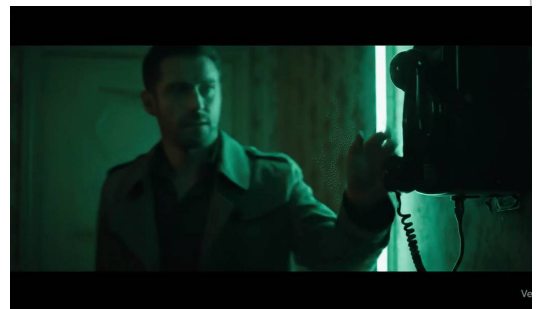
These videos demonstrate how you can revise your prompt with increasingly specific details to get Veo to refine the output to your liking.

Prompt

Generated output

Less detail

The camera dollies to show a close up of a desperate man in a green trench coat. He's making a call on a rotary-style wall phone with a green neon light. It looks like a movie scene.



More detail

A close-up cinematic shot follows a desperate man in a weathered green trench coat as he dials a rotary phone mounted on a gritty brick wall, bathed in the eerie glow of a green neon sign. The camera dollies in, revealing the tension in his jaw and the desperation etched on his face as he struggles to make the call. The shallow depth of field focuses on his furrowed brow and the black rotary phone, blurring the background into a sea of neon colors and indistinct shadows, creating a sense of urgency and isolation.



Snow leopard

Prompt	Generated output
Simple prompt: A cute creature with snow leopard-like fur is walking in winter forest, 3D cartoon style render.	
Detailed prompt: Create a short 3D animated scene in a joyful cartoon style. A cute creature with snow leopard-like fur, large expressive eyes, and a friendly, rounded form happily prances through a whimsical winter forest. The scene should feature rounded, snow-covered trees, gentle falling snowflakes, and warm sunlight filtering through the branches. The creature's bouncy movements and wide smile should convey pure delight. Aim for an upbeat, heartwarming tone with bright, cheerful colors and playful animation.	

Examples by writing elements

These examples show you how to refine your prompts by each basic element.

Subject and context

Specify the main focus (subject) and the background or environment (context).

Prompt	Generated output
An architectural rendering of a white concrete apartment building with flowing organic shapes, seamlessly blending with lush greenery and futuristic elements	

Prompt	Generated output
A satellite floating through outer space with the moon and some stars in the background.	 A detailed image of a satellite in orbit above the Earth. The satellite has a complex structure with various instruments and solar panels. The Earth's curved horizon is visible, showing blue oceans and white clouds. The background is the blackness of space with a few distant stars.

Action

Specify what the subject is doing (e.g., walking, running, or turning their head).

Prompt	Generated output
A wide shot of a woman walking along the beach, looking content and relaxed towards the horizon at sunset.	 A wide shot of a woman walking along a sandy beach at sunset. She is wearing a white dress and is looking out towards the horizon. The sun is low on the horizon, casting a warm, golden glow over the scene. The ocean waves are gently lapping at the shore.

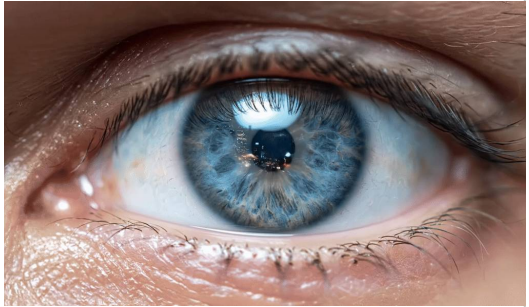
Style

Add keywords to steer the generation toward a specific aesthetic (e.g., surreal, vintage, futuristic, film noir).

Prompt	Generated output
Film noir style, man and woman walk on the street, mystery, cinematic, black and white.	 A black and white film noir style image showing a man and a woman walking down a wet, cobblestone street at night. The man is wearing a suit and hat, and the woman is wearing a dark coat. They are walking towards the camera. In the background, there are vintage cars parked on the street and buildings lining the road. The lighting is dramatic, with strong highlights and deep shadows, creating a mysterious and cinematic atmosphere.

Camera motion and composition

Specify how the camera moves (POV shot, aerial view, tracking drone view) and how the shot is framed (wide shot, close-up, low angle).

Prompt	Generated output
A POV shot from a vintage car driving in the rain, Canada at night, cinematic.	
Extreme close-up of a an eye with city reflected in it.	

Ambiance

Color palettes and lighting influence the mood. Try terms like "muted orange warm tones," "natural light," "sunrise," or "cool blue tones."

Prompt	Generated output
A close-up of a girl holding adorable golden retriever puppy in the park, sunlight.	

Prompt	Generated output
Cinematic close-up shot of a sad woman riding a bus in the rain, cool blue tones, sad mood.	

Aspect ratios

Veo lets you specify the aspect ratio for your video.

Prompt	Generated output
Widescreen (16:9) Create a video with a tracking drone view of a man driving a red convertible car in Palm Springs, 1970s, warm sunlight, long shadows.	

Prompt	Generated output
Portrait (9:16) Create a video highlighting the smooth motion of a majestic Hawaiian waterfall within a lush rainforest. Focus on realistic water flow, detailed foliage, and natural lighting to convey tranquility. Capture the rushing water, misty atmosphere, and dappled sunlight filtering through the dense canopy. Use smooth, cinematic camera movements to showcase the waterfall and its surroundings. Aim for a peaceful, realistic tone, transporting the viewer to the serene beauty of the Hawaiian rainforest.	

Model versions

Check out the [Pricing \(/gemini-api/docs/pricing#veo-3.1\)](/gemini-api/docs/pricing#veo-3.1) page and [Rate limits \(https://aistudio.google.com/rate-limit\)](https://aistudio.google.com/rate-limit) for more Veo model-specific usage details.

Veo 3.1 Preview Veo 3.1 Fast Preview... (#veo-3.1-preview)		Veo 3.1 Lite Preview...	Veo 3 (Deprecated)...
Property	Description		
 Model code	Gemini API veo-3.1-generate-preview		

 Supported data types	Input Text, Image	Output Video with audio
 Limits	Text input 1,024 tokens	Output video 1
 Latest update	January 2026	

Veo Fast versions allow developers to create videos with sound while maintaining high quality and optimizing for speed and business use cases. They're ideal for backend services that programmatically generate ads, tools for rapid A/B testing of creative concepts, or apps that need to quickly produce social media content.

What's next

- Get started with the Veo 3.1 API by experimenting in the [Veo Quickstart Colab](https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_Veo.ipynb) (https://colab.research.google.com/github/google-gemini/cookbook/blob/main/quickstarts/Get_started_Veo.ipynb) and the [Veo 3.1 applet](https://aistudio.google.com/apps/bundled/veo_studio) (https://aistudio.google.com/apps/bundled/veo_studio).
- Learn how to write even better prompts with our [Introduction to prompt design](/gemini-api/docs/prompting-intro) (/gemini-api/docs/prompting-intro).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2026-10-08 UTC.